

State-Based Alerting System — Testing Results

A plain-language record of the live tests performed and what they showed

Workflow tested: "CR drop > 10%(AJMAL)" (internal ID: wf_8ef03065fcc)

Test scenario: Alert when a product's conversion rate (CVR) drops more than 10% — marked "critical" if it drops more than 20%

Tenant / brand: AJMAL

Testing method: Running the workflow live against real store data, and directly inspecting the database after each run to confirm the system did what it was supposed to do

What "the state system" means, in one line

Instead of sending the same "your CVR is down" report every single time the workflow runs, the system now remembers each problem individually, tracks whether it is new, ongoing, getting worse, or fixed, and only emails when there is something genuinely new to say.

Summary of tests performed

#	Test	What we checked	Result
1	New finding creation	A product breaching for the first time gets tracked and flagged as "new"	Confirmed working
2	Burst cap → combined digest	When many products break at once, instead of many separate emails, they get bundled into one combined email	Confirmed working
3	Repeat-run dedup	Running the workflow again with no real change does not send another email	Confirmed working

4	State survives editing the workflow	Editing and saving the workflow (e.g. changing a threshold) does not make the system "forget" problems it already knows about	Confirmed working (after a fix — see Finding 5)
5	Resolution	A product that stops breaching for two runs in a row gets marked "resolved" and a "fixed" email goes out	Confirmed working (after a fix — see Finding 6)
6	Recurrence	A resolved product that starts breaching again is correctly treated as a fresh problem, not ignored	Confirmed working (after a fix — see Finding 6)
7	Escalation — magnitude	A product that was already flagged, then gets noticeably worse, is recognised as "getting worse," not treated as old news	Confirmed working
8	Escalation — severity tier	A product crossing from a normal drop into a "critical" drop is recognised as an escalation	Confirmed working
9	Critical bypass — cooldown	A critical-level problem is emailed immediately, even if a routine reminder for that same problem was just recently sent	Confirmed working
10	Critical bypass — quiet hours	A critical-level problem is emailed immediately, even during a configured "do not disturb" window	Confirmed working
11	Quiet hours holds non-critical alerts	A normal (non-critical) new problem, during the quiet-hours window, is held back rather than emailed right away	Confirmed working

Test details

1. New finding creation

Setup: Ran the workflow for the first time against real product data. Four products showed a CVR drop of 10% or more.

Before: No history existed for any of these products in the system.

After: All four were recorded as brand-new findings, each with its own tracking record, and the system correctly decided to notify about all four.

2. Burst cap → combined digest

Setup: The workflow was configured so that zero individual emails should go out — everything should be bundled into one email instead.

Before: All 4 breaching products were ready to notify.

After: All 4 were correctly held back from being sent as separate emails, and roughly 5 minutes later, one single email arrived listing all 4 products together.

3. Repeat-run dedup

Setup: Ran the exact same workflow again immediately, with no real change in the underlying data.

Before: The 4 products were already known, active problems.

After: No new email was sent. The system correctly recognised nothing had changed and stayed quiet, instead of repeating the same alert every run.

4. State survives editing the workflow

Setup: Edited a setting on the workflow (a threshold value) and saved it, then ran the workflow again.

Before: 4 products were being tracked as active, ongoing problems.

After the fix: Editing and saving the workflow no longer resets tracked problems back to "never seen before." The system correctly remembers them across edits.

5 & 6. Resolution and recurrence

Setup: Temporarily changed the alert condition so the same 4 products would stop counting as "breaching," then ran the workflow twice in a row (two clean readings are required before something is marked fixed).

Before: 4 products were active/breaching.

After 1 clean run: Status correctly moved to "recovering" (partway there, not yet resolved).

After 2 clean runs: Status correctly moved to "resolved," and (after a fix — see Finding 6) a "this is fixed" email was actually delivered.

Recurrence: The alert condition was then restored to normal, and the same 4 products started breaching again. The system correctly recognised this as the problem coming back, and (after the same fix) sent a notification about it rather than staying silent.

7 & 8. Escalation

Setup: Simulated two different products getting meaningfully worse than what was last reported — one by a big jump in the drop percentage, the other by crossing from a normal drop into "critical" severity.

Result: Both were correctly recognised as "escalations" (something changed for the worse), separately from an ordinary unchanged reading. The severity-crossing one correctly triggered an immediate notification; the magnitude-only one was correctly held back by the normal reminder pacing (a deliberate design choice, confirmed with the team — see note below).

Decision made during testing: A non-critical "got worse" reading is intentionally kept on the same reminder pacing as routine updates. Only crossing into critical severity forces an immediate notification.

9 & 10. Critical bypass

Setup: A product was made to freshly cross into "critical" severity, at a moment when (a) a routine reminder for it had just gone out minutes earlier, and (b) a "quiet hours" do-not-disturb window was active.

Result: The critical alert went out anyway, correctly overriding both the recent-reminder pacing and the quiet-hours window. This matches the intended behaviour — genuinely urgent problems should never get stuck waiting.

11. Quiet hours holds non-critical alerts

Setup: A "quiet hours" window was set to cover the current time. A brand-new, non-critical finding was then triggered during that window.

Result: Confirmed the notification was correctly held back rather than sent immediately, and would be released once the quiet-hours window ends.

Issues found and fixed during testing

Live testing surfaced several real problems in how the system was wired up. All were fixed and re-verified before moving to the next test. None of these were found by automated tests alone — they only showed up when actually running the system against real data.

Finding 1 — The system couldn't read a product's history at all

A required internal connection was missing, so running the workflow immediately failed with a configuration error. Nothing could be tested until this was wired up.

Finding 2 — Emails had no recipients

The list of people to email was never actually being read from the workflow. A notification would be prepared successfully but silently fail to send because it had nobody to send it to.

Finding 3 — Combined-email "batching" could get permanently stuck

Once a batch of alerts had been combined and sent once, any later alerts for the same workflow would silently pile up in a dead batch that could never be sent — with no error shown anywhere. This was fixed so a new batch always starts fresh after the previous one finishes.

Finding 4 — Same recipients problem, for the combined-email path

Similar to Finding 2, but for the "combined digest" email specifically — it also had no recipients wired in, so it would quietly fail to send and never retry.

Finding 5 — Editing the workflow reset all tracked problems

Every time the workflow was edited and saved, the system's internal "memory" of ongoing problems was tied to that exact save, rather than to the workflow itself. This meant a routine edit (like fixing a threshold) would silently make the system forget every problem it was already tracking, and treat all of them as brand new again. This is a meaningful real-world concern beyond testing — anyone editing a live alert in normal use would have hit this. Fixed so tracked problems now survive workflow edits.

Finding 6 — "Fixed" and "it's back" notifications could get silently blocked

The system has a built-in pacing rule (a "cooldown") to stop it from repeating the same reminder too often. This rule was incorrectly also applying to "this is now fixed" and "the problem is back" notifications — meaning if a problem got fixed (or came back) soon after its last reminder, the system would stay completely silent about it. Fixed so these two notification types always go through, regardless of recent reminder timing.

Finding 7 — Repeated test runs made the results screen look empty

When testing the same scenario multiple times in a row (as is normal during manual testing), the run details screen would sometimes show nothing at all, even though the system had correctly processed everything behind the scenes. This was a record-keeping issue only — the underlying decisions were always correct — but it made it hard to see what had actually happened. Fixed so every run now shows its own results clearly, even repeat runs of the same test.

Overall conclusion

Every test performed passed, and every real issue that surfaced during testing was fixed and re-confirmed with another live run. The core behaviours — recognising new problems, avoiding repeat spam, batching alerts sensibly, detecting when things get better or worse, and making sure genuinely urgent (critical) problems are never delayed — are all working as intended on real data.

This document covers only the tests that were actually performed and completed. Human-control actions (acknowledge, snooze, mute), and a few other lower-priority behaviours, have not yet been tested and are intentionally left out of this report.